

Student

My queries work correctly but they are getting slower as the table grows. Someone said I need to add indexes. What exactly is an index and why does it matter?

ChatGPT • AI Assistant

CONCEPT CARD

An **index** is a separate data structure maintained alongside a table that allows the database to jump directly to matching rows instead of reading every row from start to finish.

Without an index, every query that filters or joins on a column must perform a **full sequential scan** — reading every single row in the table regardless of how many rows actually match. With an index, the database can find matching rows in a fraction of the reads.

🌐 REAL WORLD APPLICATION

Indexes are the single most impactful performance optimization available in SQL. A query that takes 45 seconds on a 10-million-row table with a full scan can return in under 10 milliseconds with the right index — a 4,500x speedup without changing a single line of application code.

Student

What schema are we using?

ChatGPT • AI Assistant

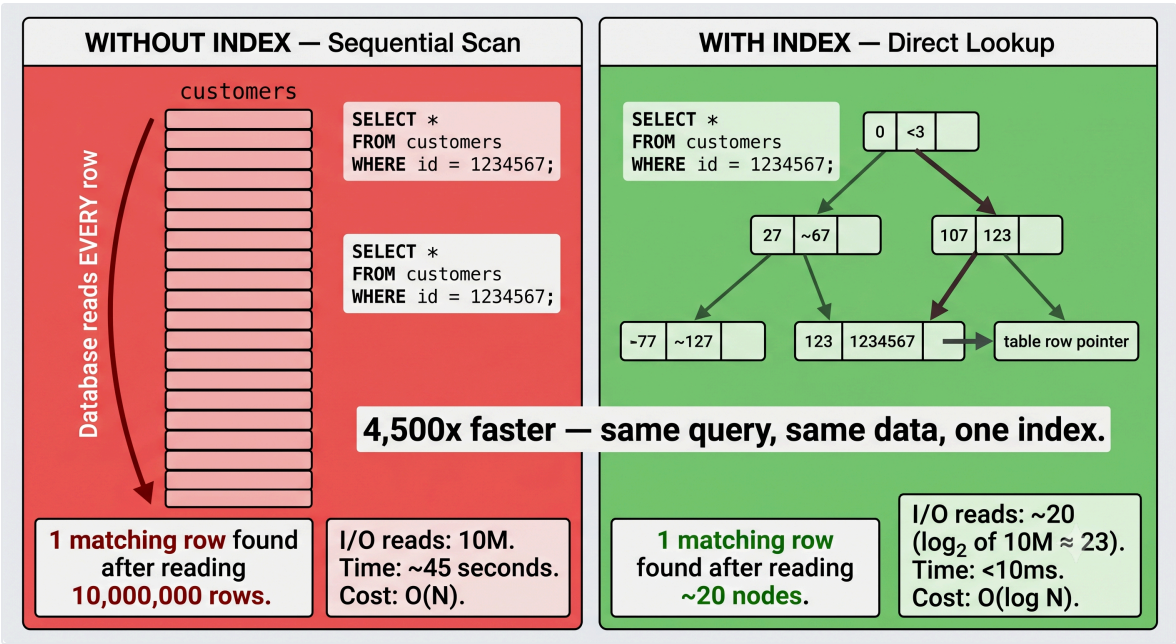
CONCEPT CARD

Same e-commerce schema from the previous sections, now focusing on performance:

- `customers(customer_id, name, email, city, signup_date)`
- `orders(order_id, customer_id, order_date, total_amount, status)`
- `products(product_id, name, category, price, stock_qty)`

- `order_items(order_id, product_id, quantity, unit_price)`

We will use realistic row counts: customers — 500,000 rows, orders — 10,000,000 rows, products — 50,000 rows.



Index Type	Internal Structure	Equality (=)	Range (>, <, BETWEEN)	Prefix LIKE ('abc%')	Suffix LIKE ('%abc')	Lo Co
	spatial/geometric data					
Partial index	B-tree on a subset of rows (WHERE clause)	Yes	Yes	Yes	No	O(M) N

sql – Index Creation Syntax Templates

```
-- Default B-tree index (most common)
CREATE INDEX idx_customers_email
  ON customers(email);

-- Unique index (enforces uniqueness + fast lookup)
CREATE UNIQUE INDEX idx_customers_email_unique
  ON customers(email);

-- Composite index (column order matters!)
CREATE INDEX idx_orders_city_date
  ON orders(customer_id, order_date);

-- Hash index (equality only)
CREATE INDEX idx_customers_email_hash
  ON customers USING HASH (email);

-- Partial index (index only a subset of rows)
CREATE INDEX idx_orders_pending
  ON orders(customer_id, order_date)
  WHERE status = 'pending';

-- Covering index (include extra columns for index-only scan)
CREATE INDEX idx_orders_covering
  ON orders(customer_id)
  INCLUDE (total_amount, status);

-- Drop an index
DROP INDEX idx_customers_email;

-- View query plan (EXPLAIN)
```

```
EXPLAIN ANALYZE
SELECT * FROM customers WHERE email = 'user@example.com';
```

THE CORE PROBLEM — FULL SEQUENTIAL SCAN AND WHY IT HURTS

ChatGPT • AI Assistant

A vertical diagram showing the customers table as a stack of rectangular page blocks numbered 1 through 100,000. Each page block contains a small icon showing ~100 rows. A red downward arrow labeled 'Sequential scan — reads ALL pages' sweeps through all page blocks. At page 87,432 (highlighted in green), a checkmark icon reads 'Match found: user@example.com'. Despite the match at page 87,432, the red arrow continues downward through the remaining pages to page 100,000, with annotation 'Must continue — may be more matches.' At the bottom, a cost summary box reads: 'Total pages read: 100,000. Total rows compared: 10,000,000. Matches found: 1. Time: ~45 seconds.' A comparison metric strip on the right shows: 'With index: reads ~23 pages. Time: <1ms.' Clean vertical stack layout, page blocks use alternating light gray and white, the match page highlighted green, red arrow spans full height.

Sequential Scan Page-by-Page Read Visualized

sql — EXPLAIN ANALYZE — Before and After Index Creation

```
-- Step 1: Check the query plan BEFORE adding an index
EXPLAIN ANALYZE
SELECT customer_id, name
FROM customers
WHERE email = 'user@example.com';

-- Output WITHOUT index:
-- Seq Scan on customers (cost=0.00..289432.00 rows=1 width=150)
--   (actual time=44821.421..44821.422 rows=1 loops=1)
--   Filter: (email = 'user@example.com')
--   Rows Removed by Filter: 9999999
--   Planning Time: 0.1 ms
--   Execution Time: 44821.5 ms ← ~45 seconds!

-- Step 2: Add the index
CREATE INDEX idx_customers_email ON customers(email);

-- Step 3: Check the query plan AFTER adding the index
EXPLAIN ANALYZE
SELECT customer_id, name
FROM customers
WHERE email = 'user@example.com';
```

```
-- Output WITH index:
-- Index Scan using idx_customers_email on customers
--   (cost=0.56..8.58 rows=1 width=150)
--   (actual time=0.043..0.044 rows=1 loops=1)
-- Index Cond: (email = 'user@example.com')
-- Planning Time: 0.2 ms
-- Execution Time: 0.1 ms ← under 1 millisecond!

-- Key metrics to look for in EXPLAIN output:
-- 'Seq Scan' = no index used (potential problem)
-- 'Index Scan' = index used for lookup + fetches table row
-- 'Index Only Scan' = index contains all needed columns (fastest)
-- 'Bitmap Index Scan' = used when many rows match (batches I/O)
```

Student

What exactly happens when there is no index and I query `WHERE email = 'user@example.com'` on a 10-million-row table?

ChatGPT • AI Assistant

CONCEPT CARD

The database performs a **full sequential scan** — it reads every row in the table from first to last, comparing each email value against your filter. This is the only option when no index exists.



$$\text{Cost}_{\{\text{sequential}\}} = O(N)$$

For a 10-million-row table, that is 10 million row reads for a query that matches exactly one row.

EXAMPLE CARD**The Cost in Numbers:**

sql — Code Example

```
-- This query on 10M rows without an index:
SELECT customer_id, name
FROM customers
WHERE email = 'user@example.com';
```

What happens internally:

- The database engine opens the customers table file on disk.
- It reads page 1 (contains ~100 rows), checks each email — no match.
- It reads page 2 — no match.
- ... repeats for all 100,000 pages ...
- It reads page 87,432 — match found. Returns the row.
- It continues reading remaining pages anyway (it does not know if more matches exist).

Cost breakdown:

code — Code Example

```
Table size:      10,000,000 rows
Rows per page:   ~100 rows (8KB page, ~80 bytes per row)
Total pages:     ~100,000 disk pages
I/O reads:       ~100,000 page reads
Typical time:    30 to 60 seconds on spinning disk
                 2 to 5 seconds on SSD (still unacceptably slow in
                 production)
```

Student

What if the same query runs 1,000 times per second on a busy API?

ChatGPT • AI Assistant

⚠ WARNING

This is exactly how production systems collapse. Each individual request takes 45 seconds. With 1,000 requests per second, the database is trying to run 45,000 concurrent sequential scans simultaneously — it melts down within seconds. This is a real pattern behind many production outages.

💡 ANALOGY

A sequential scan is like finding a phone number in a phone book by reading every single name from page 1. An index is like using the alphabetical structure — you jump to the right letter section, then the right page, and find the name in seconds. The phone book has the same data either way — the structure makes the difference.

When does the database choose a sequential scan even if an index exists?

The query optimizer compares the estimated cost of each access method:

- If a query matches a large fraction of the table (say, 30%+ of rows), a sequential scan is actually faster — random index lookups generate more I/O than a single sequential read when many rows are needed.
- The optimizer uses table statistics (row count, column cardinality, value distribution) to make this decision.
- You can see what it chose using **EXPLAIN** :

sql — Code Example

```
EXPLAIN ANALYZE
SELECT * FROM customers WHERE email = 'user@example.com';

-- Without index output:
-- Seq Scan on customers (cost=0.00..289432.00 rows=1 width=150)
--                               (actual time=44821.421..44821.422 rows=1
loops=1)
-- Filter: (email = 'user@example.com')
-- Rows Removed by Filter: 9999999

-- With index output:
-- Index Scan using idx_customers_email on customers
--                               (cost=0.56..8.58 rows=1 width=150)
--                               (actual time=0.043..0.044 rows=1 loops=1)
-- Index Cond: (email = 'user@example.com')
```

The **actual time** drops from 44,823ms to 0.044ms — a 1-million-fold improvement on this single query.

MEMORY TRICK CARD

Remember the index analogy: **I = Index = Instantly find. S = Scan = Slow sequential search.**

When you query a column with no index, the database is forced to S-can. Add an I-index and it can I-nstantly jump.

INTERVIEW CARD

Frequency: Very High

Question: What is a full sequential scan? When does the database use one even if an index exists?

Expected Answer: A full sequential scan reads every row in the table from start to finish — $O(N)$ cost. The database uses it when no index exists on the filtered column.

It also chooses a sequential scan over an index when the query is expected to return a large fraction of rows (typically 10-30%+), because random index lookups produce more total I/O than a single sequential pass when many rows are needed. The query optimizer uses table statistics to decide. You inspect this decision with EXPLAIN ANALYZE.

Follow-up: What does EXPLAIN ANALYZE show that EXPLAIN alone does not? (Answer: EXPLAIN shows the estimated plan and estimated costs. EXPLAIN ANALYZE actually executes the query and shows real execution times, real row counts, and actual vs estimated comparisons — revealing when optimizer estimates are wrong.)

🔗 PRACTICE QUIZ

1. What is the Big-O cost of a full sequential scan on a table with N rows?
2. Why does the database keep reading after finding a matching row in a sequential scan?
3. When might the query optimizer choose a sequential scan over an index even when an index exists?
4. What SQL command shows you which access method (scan vs index) the database chose for your query?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. $O(N)$ — the database must read every row regardless of how many match.
2. Because the database does not know if more rows match the filter. It must read all pages to guarantee completeness.
3. When the query is expected to return a large percentage of the table — random index lookups for many rows generate more total I/O than a single sequential pass. Also when table statistics are stale.
4. EXPLAIN ANALYZE — shows the actual execution plan including access method, estimated vs actual row counts, and real timing.

🔗 CHECKPOINT QUESTION

A `logs` table has 50 million rows. A query `SELECT * FROM logs WHERE user_id = 12345` takes 3 minutes. Another query `SELECT * FROM logs WHERE log_level = 'ERROR'` — where 40% of all rows have `log_level = 'ERROR'` — takes 4 minutes.

Would adding an index on loglevel help the second query? Explain using the optimizer's logic.

B-TREE INDEX — HOW IT WORKS INTERNALLY

ChatGPT • AI Assistant

A tree diagram showing a 4-level B-tree index on the email column of customers table. Level 0 (root node, dark blue rectangle at top): contains 3 key values dividing the space — 'g@', 'm@', 's@' — with 4 pointer arrows going down to Level 1 nodes. Level 1 (4 internal nodes, medium blue rectangles): each node contains 3-4 key values further dividing the range. Level 2 (internal nodes, light blue rectangles): 16 nodes continuing the subdivision. Level 3 (leaf nodes, green rectangles at bottom): actual email values paired with '→ page X, slot Y' pointers. The leaf nodes are connected left-to-right by a horizontal chain arrow labeled 'Leaf nodes linked in sorted order — enables range scans.' A highlighted path in orange traces the search for 'r@example.com' from root through specific internal nodes to the correct leaf node — 4 nodes touched total. A counter badge reads: '4 node reads vs 100,000 page reads for sequential scan.' Below the tree, a supported operations box lists: 'Equality ✓, Range ✓, Prefix LIKE ✓, Suffix LIKE X.' Clean hierarchical tree layout, color-coded levels.

B-Tree Index Structure — Root to Leaf Traversal

sql — B-Tree Index — Supported and Unsupported Query Patterns

```
-- CREATE the B-tree index (default type)
CREATE INDEX idx_customers_email      ON customers(email);
CREATE INDEX idx_orders_order_date   ON orders(order_date);
CREATE INDEX idx_products_price      ON products(price);

-- SUPPORTED: Equality lookup → O(log N)
SELECT * FROM customers WHERE email = 'user@example.com';

-- SUPPORTED: Range query → find start, scan leaf chain
SELECT * FROM orders WHERE order_date > '2024-01-01';
SELECT * FROM products WHERE price BETWEEN 500 AND 2000;
SELECT * FROM orders WHERE total_amount ≥ 1000;

-- SUPPORTED: Prefix LIKE → find 'Amit' range, scan forward
SELECT * FROM customers WHERE name LIKE 'Amit%';

-- SUPPORTED: ORDER BY on indexed column → already sorted, no sort step
SELECT * FROM customers ORDER BY email LIMIT 100;

-- NOT SUPPORTED: Suffix LIKE → full scan despite index
SELECT * FROM customers WHERE name LIKE '%Kumar';

-- NOT SUPPORTED: Substring LIKE → full scan
SELECT * FROM customers WHERE name LIKE '%Amit%';
```

```
-- NOT SUPPORTED: Function on indexed column → index bypassed
SELECT * FROM orders WHERE EXTRACT(YEAR FROM order_date) = 2024;
-- Fix: use range instead:
SELECT * FROM orders
WHERE order_date ≥ '2024-01-01' AND order_date < '2025-01-01';

-- NOT SUPPORTED: Implicit type cast → index bypassed
SELECT * FROM customers WHERE customer_id = '101'; -- string vs INT
column
-- Fix: use matching type:
SELECT * FROM customers WHERE customer_id = 101;
```

Student

What exactly is a B-tree index? How does the database use it to find a row so fast?

ChatGPT • AI Assistant

CONCEPT CARD

A **B-tree** (Balanced Tree) index is a sorted tree data structure. It has three levels:

- **Root node** — the top entry point, holds key ranges that direct the search down
- **Internal nodes** — intermediate levels, narrow down the search range further
- **Leaf nodes** — the bottom level, hold actual key values paired with pointers to the matching table rows

Every path from root to leaf has the same length — that is what 'balanced' means. This guarantees $O(\log N)$ lookup regardless of which value you search for.

$$\text{Lookup Cost} = O(\log_B N)$$

Where B is the branching factor (typically 100-500 in real databases) and N is the number of indexed rows. For 10 million rows with branching factor 200:

$$\text{Levels} = \lceil \log_{200}(10,000,000) \rceil = 4; \text{levels}$$

The database reads exactly 4 nodes to find any value — regardless of which value, regardless of where it sits in the table.

EXAMPLE CARD

Step-by-Step B-tree Traversal: Find email = 'r@example.com'

Imagine the index is built on the email column of customers (10M rows).

code — Code Example

Root Node:

```
[ 'h@... ' | 'm@... ' | 's@... ' ]  
  left      center   right
```

Step 1: 'r@... ' > 'm@... ' and 'r@... ' < 's@... ' → go to center child

Internal Node (m to s range):

```
[ 'n@... ' | 'q@... ' | 'r5@... ' ]
```

Step 2: 'r@... ' > 'q@... ' and 'r@... ' < 'r5@... ' → go to center-right child

Leaf Node:

```
[ 'r@example.com' → page 87432, slot 14 ]  
[ 'r1@example.com' → page 12001, slot 3  ]  
[ 'ra@example.com' → page 54231, slot 7  ]
```

Step 3: Match found — 'r@example.com' → page 87432, slot 14

Step 4: Database fetches page 87432 from disk, reads slot 14 → returns row

Total disk reads: 4 (root + 2 internal nodes + leaf + 1 table page) = **5 page reads** instead of 100,000.

Student

What types of queries can a B-tree index support?

ChatGPT • AI Assistant

CONCEPT CARD

B-tree indexes work for **equality, range, and prefix pattern** queries — because the tree is sorted:

Equality:

sql – Code Example

```
WHERE email = 'user@example.com'    -- jumps directly to leaf node
```

Range queries:

sql – Code Example

```
WHERE signup_date > '2024-01-01'    -- find start point, scan leaf
nodes right
WHERE price BETWEEN 500 AND 2000    -- find lower bound, scan to upper
bound
WHERE total_amount ≥ 1000           -- find first match, scan forward
```

Leaf nodes in a B-tree are linked in sorted order — after finding the start of a range, the database just scans forward through the leaf node chain.

Prefix LIKE:

sql – Code Example

```
WHERE name LIKE 'Amit%'    -- B-tree can find prefix 'Amit' and scan
forward
```

NOT supported:

sql – Code Example

```
WHERE name LIKE '%Kumar'    -- suffix match – no way to start the tree
traversal
WHERE name LIKE '%Kumar%'    -- substring match – same problem
```

Suffix and substring matches cannot use a B-tree because there is no way to determine where in the sorted tree to begin the search.

❗ COMMON PITFALL / MISTAKE

✗ Expecting LIKE '%keyword' to use an index:

sql – Code Example

```
SELECT * FROM customers WHERE name LIKE '%Kumar';
-- Full sequential scan even if index exists on name
```

- The % at the START means: 'could start with anything'
- The B-tree has no entry point for this search

✅ Prefix match uses the index:

sql – Code Example

```
SELECT * FROM customers WHERE name LIKE 'Kumar%';  
-- Uses B-tree index: find 'Kumar' range and scan forward
```

MEMORY TRICK CARD

Remember: **'Starts-with' uses the index. 'Ends-with' does not.**

LIKE 'abc%' = B-tree friendly. LIKE '%abc' = full scan.

Think of the B-tree as a dictionary — you can find words that start with 'abc' instantly. Finding all words that end with 'abc' means reading every page.

🔗 INTERVIEW CARD

Frequency: Very High

Question: Explain how a B-tree index works. What types of queries does it support?

Expected Answer: A B-tree is a balanced sorted tree with root, internal, and leaf nodes. The leaf nodes contain the actual indexed values paired with pointers to the matching table rows. A lookup traverses from root to leaf following key comparisons — $O(\log N)$ reads regardless of table size. Leaf nodes are linked in sorted order, enabling efficient range scans. B-tree indexes support equality (=), range (>, <, >=, <=, BETWEEN), and prefix LIKE ('abc%'). They do NOT support suffix LIKE ('%abc') or substring LIKE ('%abc%') because there is no traversal entry point for those patterns.

Follow-up: Why is a B-tree called 'balanced'? (Answer: Every path from root to leaf node has the same length — the tree rebalances itself automatically on insertions and deletions to maintain this property. This guarantees consistent $O(\log N)$ performance regardless of which value is searched.)

🔗 PRACTICE QUIZ

1. How many disk reads does a B-tree index need to find a value in a 10-million-row table?
2. Does a B-tree index on `name` help the query `WHERE name LIKE '%Smith'` ?
3. Why can B-tree indexes support range queries but hash indexes cannot?

4. What is the mathematical formula for the number of B-tree levels needed to index N rows with branching factor B?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

- 1. Approximately $\log_B(N)$ reads — for 10M rows with branching factor 200, approximately 4 node reads plus 1 table page fetch = ~5 total disk reads.
- 2. No — the leading wildcard % means the pattern can start with any character. The B-tree has no entry point for suffix searches. Full scan required.
- 3. B-tree leaf nodes are linked in sorted order — after finding the range start point, the database scans forward through the linked leaf chain. Hash indexes map keys to buckets with no ordering — there is no concept of 'next value' in a hash structure.
- 4. `Levels = ceil(log_B(N))`

🔍 CHECKPOINT QUESTION

You have an `orders` table with 10 million rows and an index on `order_date` . Explain step by step how the database executes `SELECT * FROM orders WHERE order_date BETWEEN '2024-01-01' AND '2024-01-31'` using the B-tree index. How is this different from an equality lookup?

HASH INDEX — EQUALITY-ONLY O(1) LOOKUPS

ChatGPT • AI Assistant

A two-part diagram. Top half (label 'Hash Index Equality Lookup — O(1)'): shows the query value 'user@example.com' entering a hash function box labeled 'hash() mod 10,000'. An arrow exits to 'Bucket 3' highlighted in green. From Bucket 3, an arrow points to 'Row pointer → page 87432.' A performance badge reads: 'O(1) — same speed for 1 row or 1 billion rows.' Bottom half (label 'Hash Index Range Query — FAILS'): shows the query 'price BETWEEN 500 AND 2000'. A hash function box maps '500' to 'Bucket 7,231' and '2000' to 'Bucket 3,814'. A large red X and annotation reads: 'No way to know which buckets contain values between 500 and 2000. All buckets must be checked = full scan.' Comparison summary at the bottom: a two-column table with rows for 'Equality =', 'Range >', 'BETWEEN', 'PREFIX LIKE', 'ORDER BY' — B-tree column shows green checks for all, Hash column shows green check for equality only, red X for all others. Clean diagram, consistent color coding.

Hash Index — Bucket Mapping and Range Query Limitation

sql — Hash Index — Creation, Equality Wins, Range Failures

```

-- Create a hash index (PostgreSQL syntax)
CREATE INDEX idx_customers_email_hash
    ON customers USING HASH (email);

-- WORKS: equality lookup O(1)
SELECT customer_id, name
FROM customers
WHERE email = 'user@example.com';
-- Uses hash index: hash('user@..') → bucket → row

-- FAILS to use hash index: range query
SELECT * FROM orders WHERE total_amount > 1000;
-- Falls back to sequential scan or another index

-- FAILS to use hash index: prefix LIKE
SELECT * FROM customers WHERE email LIKE 'user%';
-- Hash cannot resolve prefix ranges

-- FAILS to use hash index: ORDER BY
SELECT * FROM customers ORDER BY email;
-- Hash has no sorted order to exploit

-- Decision: when to use hash vs B-tree
-- Column: session_token (UUID) – equality only, high cardinality
CREATE INDEX idx_sessions_token_hash
    ON sessions USING HASH (session_token); -- hash: fine here

-- Column: order_date – range queries needed
CREATE INDEX idx_orders_date_btree
    ON orders(order_date); -- B-tree: default, supports ranges

-- EXPLAIN to compare actual plans
EXPLAIN ANALYZE SELECT * FROM customers WHERE email = 'test@test.com';
-- Look for: 'Index Scan using idx_customers_email_hash'
-- vs:      'Seq Scan on customers'

```

Student

You mentioned hash indexes. If they are $O(1)$ for equality, why would I ever use B-tree instead?

CONCEPT CARD

A **hash index** applies a hash function to the key value, maps it to a bucket number, and stores the row pointer in that bucket. Lookup is $O(1)$ for equality — independent of table size.

$$\text{bucket} = \text{hash}(\text{key}) \ \backslash \text{mod} \ \text{num_buckets}$$
🔗 ANALOGY

A hash index is like a coat check at a venue. You hand over your coat and get a numbered ticket. When you return, you give the ticket number and instantly get your coat back — $O(1)$. But if you ask 'which coats belong to people whose last name starts with K?' the coat check is useless — the numbering system has no alphabetical order.

EXAMPLE CARD**How a hash index lookup works:**

code — Code Example

Query: `WHERE email = 'user@example.com'`

Step 1: `hash('user@example.com') = 4,829,103`

Step 2: `bucket = 4,829,103 mod 10,000 = bucket 3`

Step 3: Go directly to bucket 3

Step 4: Read pointer → page 87432, slot 14

Step 5: Fetch row from page 87432

Total reads: 2 (bucket + table page)

Cost: $O(1)$ — same speed whether table has 1 row or 1 billion rows

Creating a hash index (PostgreSQL):

sql — Code Example

```
CREATE INDEX idx_customers_email_hash
ON customers USING HASH (email);
```

That sounds amazing — always $O(1)$. Why is this not the default?

ChatGPT • AI Assistant

Because hash indexes have **critical limitations** that make them unsuitable for most real queries:

What hash indexes CANNOT do:

sql – Code Example

```
-- FAILS: range query
SELECT * FROM orders WHERE total_amount > 1000;
-- Hash has no ordering – cannot determine which bucket '> 1000' maps
to

-- FAILS: prefix LIKE
SELECT * FROM customers WHERE name LIKE 'Amit%';
-- Hash of 'Amit' tells you nothing about 'Amitabh', 'Amita', etc.

-- FAILS: ORDER BY optimization
SELECT * FROM customers ORDER BY email;
-- Hash has no sorted structure – cannot skip the sort step

-- FAILS: BETWEEN
SELECT * FROM products WHERE price BETWEEN 500 AND 2000;
-- Cannot determine range boundaries from hash buckets
```

What hash indexes CAN do:

sql – Code Example

```
-- WORKS: exact equality only
SELECT * FROM customers WHERE email = 'user@example.com'; -- O(1)
SELECT * FROM orders WHERE status = 'pending';           -- O(1)
```

⚠ WARNING

In PostgreSQL before version 10, hash indexes were not WAL-logged — they were not crash-safe and were lost on database crash. From PostgreSQL 10+ they are safe. In MySQL, InnoDB (the default storage engine) does NOT support manual hash indexes — it uses adaptive hash indexes internally and automatically. MySQL only supports hash indexes explicitly in the MEMORY storage engine.

CONCEPT CARD**The Decision Rule — B-tree vs Hash:**

- Use **B-tree** as the default for all columns. It handles equality, range, LIKE prefix, and ORDER BY.
- Use **Hash** only when ALL of these are true:
 3. The column is used ONLY for equality comparisons — never range, never ORDER BY.
 4. The column has high cardinality (many distinct values — like a UUID or email).
 5. You are using PostgreSQL 10+ (where hash indexes are crash-safe).
 6. You have benchmarked and confirmed the $O(1)$ improvement is meaningful.

In practice, B-tree is the right choice 95%+ of the time.

 INTERVIEW CARD

Frequency: Medium-High

Question: What is a hash index? When would you use it instead of a B-tree?

Expected Answer: A hash index applies a hash function to the key and maps it to a bucket containing row pointers. Equality lookups are $O(1)$ — independent of table size. However, hash indexes cannot support range queries ($>$, $<$, BETWEEN), prefix LIKE, or ORDER BY optimization because the hash function destroys value ordering. Use a hash index only for high-cardinality columns that are exclusively used for equality comparisons and never for ranges. B-tree handles both equality and range and should be the default for almost all cases.

Follow-up: Does MySQL InnoDB support explicit hash indexes? (Answer: No — InnoDB manages adaptive hash indexes internally and automatically. Manual hash indexes in MySQL are only available with the MEMORY storage engine.)

 PRACTICE QUIZ

1. What is the lookup cost of a hash index for an equality query?
2. Can a hash index be used for `WHERE price BETWEEN 500 AND 2000` ? Why or why not?
3. Name three query types that a B-tree supports but a hash index does not.
4. Predict: you have `CREATE INDEX idx_hash ON orders USING HASH (status)` . The query `SELECT * FROM orders WHERE status > 'pending'` runs. Does it use the index?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. $O(1)$ — the hash function maps directly to the bucket regardless of table size.
2. No — the hash function maps each value to a bucket with no preserved ordering. The database cannot determine which buckets contain values between 500 and 2000 without checking every bucket.
3. Any three of: range queries ($>$, $<$, BETWEEN), prefix LIKE, ORDER BY optimization, multi-column range scans.
4. No — the hash index is not used for comparison operators other than $=$. The database falls back to a sequential scan.

❓ CHECKPOINT QUESTION

You have a `sessions` table with 200 million rows. Columns: `sessionId` (UUID), `userId` (INT), `createdAt` (TIMESTAMP), `lastactive` (TIMESTAMP), `is_expired` (BOOLEAN).

Which column(s) would benefit most from a hash index? Which should use a B-tree? Justify each decision.

COMPOSITE INDEXES, THE LEFTMOST PREFIX RULE, AND COVERING INDEXES

ChatGPT • AI Assistant

A three-panel diagram illustrating `INDEX(customer_id, order_date)`. Left panel (label '`INDEX(customer_id, order_date)` structure'): shows a sorted data representation — rows grouped by `customer_id` (101, 102, 103) with `order_date` sorted within each group. Bracket annotations show '`customer_id = 101` → contiguous block' highlighted green, '`customer_id = 102` → contiguous block' highlighted blue. Top panel (green background, label '`USES INDEX`'): three query boxes each with a green checkmark: (1) `WHERE customer_id = 101`, (2) `WHERE customer_id = 101 AND order_date > '2024-06-01'`, (3) `ORDER BY customer_id, order_date`. Bottom panel (red background, label '`DOES NOT USE INDEX EFFICIENTLY`'): two query boxes with red X marks: (1) `WHERE order_date > '2024-06-01'` — annotation '`order_date` values scattered across all customer groups'; (2) `WHERE order_date BETWEEN '2024-01-01' AND '2024-03-31'` — same annotation. A rule box reads: 'Leftmost prefix rule: must filter on leftmost column first. Think: phone book sorted by last name then first name.' Clean three-panel layout, color-coded query boxes.

Composite Index Leftmost Prefix Rule — What Works and What Does Not

sql – Composite Index, Leftmost Prefix Rule, and Covering Index

```
-- Composite index: customer_id THEN order_date
CREATE INDEX idx_orders_cust_date
    ON orders(customer_id, order_date);

-- USES composite index (leftmost column present)
```

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 101;
-- Index Scan using idx_orders_cust_date

EXPLAIN SELECT * FROM orders
WHERE customer_id = 101 AND order_date > '2024-01-01';
-- Index Scan using idx_orders_cust_date

-- DOES NOT use composite index efficiently (leftmost column absent)
EXPLAIN SELECT * FROM orders WHERE order_date > '2024-01-01';
-- Seq Scan on orders OR uses a different index if one exists on
order_date

-- Composite index on customers: city + signup_date
CREATE INDEX idx_customers_city_date
    ON customers(city, signup_date);

EXPLAIN SELECT * FROM customers WHERE city = 'Delhi';
-- Uses index: city is leftmost ✓

EXPLAIN SELECT * FROM customers
WHERE city = 'Delhi' AND signup_date > '2024-01-01';
-- Uses both columns of index ✓

EXPLAIN SELECT * FROM customers WHERE signup_date > '2024-01-01';
-- Does NOT use composite index: signup_date is not leftmost X

-- Covering index: query never touches base table
CREATE INDEX idx_orders_covering
    ON orders(customer_id)
    INCLUDE (total_amount, status);

EXPLAIN SELECT customer_id, total_amount, status
FROM orders WHERE customer_id = 101;
-- Index Only Scan using idx_orders_covering
-- All needed columns in index leaf – no table fetch!

-- Optimal covering composite index for a specific query
CREATE INDEX idx_customers_city_signup_cover
    ON customers(city, signup_date DESC)
    INCLUDE (name, email);
-- Supports: WHERE city='Delhi' AND signup_date > '2024-01-01'
-- ORDER BY signup_date DESC
-- SELECT name, email (covered – no table fetch)
```

What is a composite index? Is it just an index on two columns?

ChatGPT • AI Assistant

CONCEPT CARD

A **composite index** (also called a multi-column index) is a single index built on two or more columns together. The B-tree sorts rows first by the first column, then by the second column within each first-column group, then by the third, and so on.

sql – Code Example

```
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, order_date);
```

This index stores rows sorted by **customer_id** first, then **order_date** within each customer. It is like a phone book sorted by last name, then first name within each last name.

CONCEPT CARD

The Leftmost Prefix Rule:

A composite index on (**col_a**, **col_b**, **col_c**) can be used by queries that filter on:

- **col_a** alone
- **col_a** AND **col_b**
- **col_a** AND **col_b** AND **col_c**

It CANNOT be used (efficiently) by queries that filter on:

- **col_b** alone (skips the leftmost column)
- **col_c** alone
- **col_b** AND **col_c** (leftmost column missing)

EXAMPLE CARD

Example 1 — Composite index on (customer_id, order_date):

sql – Code Example

```
CREATE INDEX idx_orders_cust_date ON orders(customer_id, order_date);

-- USES the index: customer_id is leftmost
SELECT * FROM orders WHERE customer_id = 101;
```

```
-- USES the index: both columns, leftmost first
SELECT * FROM orders
WHERE customer_id = 101 AND order_date > '2024-01-01';

-- PARTIALLY uses the index: customer_id filter uses it,
-- but cannot use the date part of the index alone
SELECT * FROM orders WHERE order_date > '2024-01-01';
-- Result: index scan on customer_id range ONLY, or full scan
-- The database cannot jump to 'all rows where order_date > X'
-- without first knowing the customer_id
```

Example 2 — Concrete illustration of WHY order matters:

code – Code Example

Composite index (customer_id, order_date) stores data as:

```
customer_id=101, order_date=2024-01-05
customer_id=101, order_date=2024-03-12
customer_id=101, order_date=2024-07-22
customer_id=102, order_date=2024-02-11
customer_id=102, order_date=2024-08-30
customer_id=103, order_date=2024-01-01
...
```

```
WHERE customer_id = 101 → jump directly to customer 101 section
WHERE customer_id = 101 AND order_date > '2024-06-01' → jump to 101,
scan forward
WHERE order_date > '2024-06-01' → rows with that date are scattered
across ALL customer groups – no contiguous section to jump to
```

🔗 ANALOGY

A composite index is like a filing cabinet sorted by last name, then first name. Finding everyone named 'Smith' is instant. Finding everyone named 'Smith' born in 1990 is also fast. Finding everyone born in 1990 regardless of last name — you must check every drawer, because birth year is the second sort key and there is no shortcut.

Student

What is a covering index? I heard it can make queries even faster.

CONCEPT CARD

A **covering index** is an index that contains ALL the columns a query needs — so the database never needs to touch the base table at all. This is called an **index-only scan** and is the fastest possible access pattern.

sql – Code Example

```
-- Query needs: customer_id, total_amount, status
SELECT customer_id, total_amount, status
FROM orders
WHERE customer_id = 101;

-- Regular index on customer_id:
-- Step 1: index lookup → find row pointers for customer 101
-- Step 2: fetch each table row to get total_amount and status

-- Covering index (PostgreSQL INCLUDE syntax):
CREATE INDEX idx_orders_covering
  ON orders(customer_id)
  INCLUDE (total_amount, status);
-- Step 1: index lookup → find leaf nodes for customer 101
-- Step 2: read total_amount and status directly from index leaf
-- NEVER touches the base table
```

INCLUDE vs extra key columns:**sql – Code Example**

```
-- Option A: compound index (status is part of the sort key)
CREATE INDEX idx_a ON orders(customer_id, total_amount, status);
-- status participates in sorting – useful for filtering/ordering on status

-- Option B: covering with INCLUDE (status is payload, not sort key)
CREATE INDEX idx_b ON orders(customer_id) INCLUDE (total_amount, status);
-- status is stored in leaf nodes for retrieval, not for sorting
-- Better when you only need status for SELECT, not for WHERE/ORDER BY
```

 INTERVIEW CARD**Frequency:** High

Question: Explain the leftmost prefix rule for composite indexes. Give a concrete example.

Expected Answer: A composite index on (*cola*, *colb*, *colc*) is a B-tree sorted first by *cola*, then *colb* within each *cola* group, then *colc* within each *colb* group. A query can use the index only if it filters on columns starting from the leftmost — *cola* alone, *cola+colb*, or all three. A query filtering only on *colb* or *colc* cannot use the index because those columns are not contiguously sorted at the top level of the tree. For example, `INDEX(city, signupdate)` helps `WHERE city = 'Delhi'` but not `WHERE signup_date > '2024-01-01'` alone — dates for that range are scattered across all city groups with no contiguous section to traverse.

Follow-up: What is a covering index and what is the performance benefit? (Answer: A covering index contains all columns the query needs in the `SELECT`, `WHERE`, and `ORDER BY`. The database performs an index-only scan — it reads the answer directly from the index leaf nodes without fetching any rows from the base table. This eliminates the second round of I/O entirely.)

🔗 PRACTICE QUIZ

1. `INDEX(city, signupdate)` — does `'WHERE signupdate > '2024-01-01'` use this index efficiently?
2. What is the leftmost prefix rule in one sentence?
3. What is the difference between a composite index column and an `INCLUDE` column in a covering index?
4. What does 'Index Only Scan' mean in an `EXPLAIN` output?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. No — `signup_date` is the second column. Without filtering on `city` (the leftmost), the database cannot use the composite index efficiently. It falls back to a sequential scan or a different index.
2. A composite index (*cola*, *colb*) can only be used by queries that filter on *cola* or on *cola* AND *colb* — never on *colb* alone.
3. A composite index column is part of the sort key and can be used in `WHERE`/`ORDER BY` filtering. An `INCLUDE` column is stored in the leaf nodes as payload for covering purposes only — it is not part of the sort key and cannot be used in `WHERE` or `ORDER BY`.

4. 'Index Only Scan' means the database read all required data directly from the index leaf nodes without touching the base table — the index covers all columns needed by the query. Fastest possible access pattern.

❓ CHECKPOINT QUESTION

You have a query: `SELECT name, email FROM customers WHERE city = 'Delhi' AND signup_date > '2024-01-01' ORDER BY signup_date DESC LIMIT 50;` Design the optimal index for this query. Should it be a composite index, a covering index, or both? Write the `CREATE INDEX` statement and explain every decision.

OVER-INDEXING, THE WRITE COST, AND THE INDEXING DECISION FRAMEWORK

ChatGPT • AI Assistant

A two-column decision matrix diagram. Left column (green background, header 'ADD AN INDEX'): five rows with green checkmark icons. Row 1: 'High-cardinality WHERE column — email, user_id, order_id.' Row 2: 'Foreign key column — orders.customer_id, order_items.order_id.' Row 3: 'JOIN ON column — frequently joined, large tables.' Row 4: 'ORDER BY column combined with selective WHERE.' Row 5: 'Covering index for a critical hot query — eliminates table fetch.' Right column (red background, header 'DO NOT INDEX'): five rows with red X icons. Row 1: 'Boolean columns — is_active, is_paid. Returns 50% of table.' Row 2: 'Low-cardinality status columns — 5-10 distinct values.' Row 3: 'SELECT-only columns not in WHERE/JOIN/ORDER BY.' Row 4: 'Tiny tables under 1,000 rows — seq scan is faster.' Row 5: 'Duplicate or redundant indexes — (a) is redundant if (a,b) exists.' Below the matrix, a process strip reads: '1. Confirm slow query with EXPLAIN ANALYZE. 2. Identify unindexed WHERE/JOIN columns. 3. Create index. 4. Re-run EXPLAIN ANALYZE to confirm. 5. Monitor pg_stat_user_indexes for unused indexes.' Clean two-column matrix layout, green/red color scheme, checkmark and X icons.

Indexing Decision Framework — When to Add and When to Avoid

```
sql – Over-Indexing Cost, Index Audit, and Decision Framework in Practice

-- Write cost demonstration
-- A table with many indexes makes inserts/updates expensive

-- Check all indexes on orders table (PostgreSQL)
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'orders'
ORDER BY indexname;

-- Find unused indexes (candidates for dropping)
SELECT schemaname,
       tablename,
       indexname,
       idx_scan      AS times_used,
       pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
```

```
FROM pg_stat_user_indexes
WHERE schemaname = 'public'
    AND idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;

-- LOW CARDINALITY: avoid indexing booleans/status alone
-- BAD (optimizer often ignores):
CREATE INDEX idx_orders_is_paid ON orders(is_paid); -- 2 distinct
values
CREATE INDEX idx_orders_status ON orders(status); -- 5 distinct
values

-- GOOD: index high-cardinality columns
CREATE INDEX idx_orders_customer_id ON orders(customer_id); -- 500k
values
CREATE INDEX idx_orders_order_date ON orders(order_date); -- many
distinct dates
CREATE INDEX idx_customers_email ON customers(email); --
unique values

-- GOOD: composite where first column is high-cardinality
CREATE INDEX idx_orders_cust_status
    ON orders(customer_id, status);
-- WHERE customer_id = 101 AND status = 'pending' → uses both columns

-- DROP unused or redundant indexes
DROP INDEX idx_orders_is_paid;
DROP INDEX idx_orders_status;

-- PARTIAL INDEX: index only the rows you actually query
CREATE INDEX idx_orders_pending_cust
    ON orders(customer_id, order_date)
    WHERE status = 'pending';
-- Smaller index, only covers pending orders
-- Helps: WHERE status = 'pending' AND customer_id = 101
-- Useless for delivered/cancelled orders
```

Student

If indexes make reads faster, why not just index every column?

⚠ WARNING

That is the most common indexing mistake. Every index you add makes **writes slower** — INSERT, UPDATE, and DELETE must update every index on the table, not just the base table row.

CONCEPT CARD**The Write Cost of Indexes:**

When you INSERT a row into a table with 8 indexes:

1. Write the new row to the table (1 write)
2. Insert the new key into index 1's B-tree and rebalance if needed
3. Insert the new key into index 2's B-tree
4. ... repeat for all 8 indexes ...
5. Total: 9 write operations instead of 1



$$\text{Write\;Cost} = O(k \times \log N)$$

Where k = number of indexes and N = table size. For a table with 10 indexes and 10 million rows, each INSERT requires $\sim 10 \times 23 = 230$ B-tree node operations.

EXAMPLE CARD**Example — The insert performance collapse:**

sql — Code Example

```
-- Table with 0 indexes:
INSERT INTO orders ( ... ) VALUES ( ... ); -- ~1ms per insert
-- Bulk load of 1M rows: ~17 minutes

-- Same table with 8 indexes:
INSERT INTO orders ( ... ) VALUES ( ... ); -- ~8ms per insert
-- Bulk load of 1M rows: ~2.2 hours
-- 8x slower per write
```

UPDATE is especially expensive:

sql — Code Example

```
UPDATE orders SET status = 'delivered', total_amount = 1850
WHERE order_id = 1001;
```

If both `status` and `total_amount` are indexed:

- Delete old key from status index
- Insert new key into status index
- Delete old key from total_amount index
- Insert new key into total_amount index
- Update the base table row

= 5 operations for 1 logical update

CONCEPT CARD

The Indexing Decision Framework — When to Add an Index:

✓ Good candidates for indexes:

- Columns used in `WHERE` clauses on large tables
- Columns used in `JOIN ON` conditions (FK columns especially)
- Columns used in `ORDER BY` when combined with a selective WHERE filter
- High-cardinality columns (many distinct values — email, order_id, timestamp)
- Foreign key columns in child tables (orders.customer_id, orderitems.order_id)

✗ Poor candidates — do NOT index these:

- **Low-cardinality columns** like `is_active (BOOLEAN)`, `status` (5 values), `gender` — an index on a boolean halves the table at best; a sequential scan is often faster
- **Rarely queried columns** — index maintenance cost on writes with no read benefit
- **Very small tables** (< 1,000 rows) — sequential scan is faster than index overhead
- **Columns only used in SELECT output** (not in WHERE, JOIN, ORDER BY)
- **Duplicate indexes** — an index on (a) is redundant if you already have (a, b)

The Low-Cardinality Rule Explained:

sql — Code Example

```
-- BAD: index on boolean column (only 2 distinct values)
CREATE INDEX idx_orders_is_paid ON orders(is_paid);
-- WHERE is_paid = true returns ~50% of rows
-- Sequential scan on 50% of 10M rows is faster than
-- 5M random index lookups!
-- The optimizer will likely ignore this index entirely

-- BAD: index on status with 5 values
CREATE INDEX idx_orders_status ON orders(status);
-- WHERE status = 'pending' returns ~20% of rows
```

```
-- Optimizer may still prefer sequential scan

-- GOOD: composite index where status filters a small subset
CREATE INDEX idx_orders_status_date
  ON orders(customer_id, status);
-- WHERE customer_id = 101 AND status = 'pending'
-- customer_id is high cardinality → small result set → index useful
```

Always use EXPLAIN before and after:

sql – Code Example

```
-- Before adding index: check if query actually does a seq scan
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 101;

-- After adding index: verify optimizer uses it
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 101;
-- Look for: Index Scan / Index Only Scan
-- Check: actual time improvement
-- If still seq scan: optimizer decided index is not worth it
```

Student

Is there a way to see all indexes on a table?

ChatGPT • AI Assistant

sql – Code Example

```
-- PostgreSQL: list all indexes on a table
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'orders';

-- MySQL: show indexes
SHOW INDEXES FROM orders;

-- Find potentially unused indexes (PostgreSQL)
SELECT schemaname, tablename, indexname,
       idx_scan, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes
```

```
WHERE idx_scan = 0      -- never used since last stats reset
ORDER BY tablename, indexname;
```

INTERVIEW CARD

Frequency: Very High

Question: What are the downsides of adding too many indexes? What is your decision framework for indexing?

Expected Answer: Every index slows down INSERT, UPDATE, and DELETE because the database must update all indexes in sync with the base table. An INSERT into a table with 8 indexes requires 8 additional B-tree write operations. The decision framework: index columns used in WHERE, JOIN ON, and ORDER BY on large tables with high cardinality. Avoid indexing low-cardinality columns (booleans, status fields with few values) — the optimizer often ignores them because reading 30% of a table via random index lookups is slower than a sequential scan. Always verify with EXPLAIN ANALYZE before and after. Periodically audit indexes using `pgstatuser_indexes` to find indexes with zero scans.

Follow-up: How does an index on a foreign key column improve JOIN performance? (Answer: Without an index on the FK column, a JOIN requires a sequential scan of the child table for each parent row. With an index, the database can directly look up child rows for each parent — $O(\log N)$ instead of $O(N)$ per parent row.)

PRACTICE QUIZ

1. A table has 10 indexes. How many write operations does a single INSERT require?
2. Why is indexing a boolean column usually a bad idea?
3. What query should you always run before and after adding an index?
4. You notice `idx_orders_status` has `idx_scan = 0` in `pgstatuser_indexes`. What does this mean and what should you do?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

ANSWER KEY

1. 11 write operations: 1 to insert the base row + 1 per index to update each B-tree = $10 + 1 = 11$.
2. A boolean has only 2 distinct values. A query like `WHERE is_active = true` may return 50% of the table. Random index lookups for 50% of rows generate more

total I/O than a single sequential scan — the optimizer ignores the index. Index maintenance cost with zero read benefit.

3. `EXPLAIN ANALYZE query` — before adding the index to confirm a sequential scan is occurring, and after to confirm the index is being used and the actual execution time improved.
4. `idx_scan = 0` means this index has never been used since the last statistics reset. It is consuming write overhead with zero read benefit. Strong candidate for dropping — but verify the statistics window is large enough (reset recently means the data may be misleading) and check if any rare but critical queries use it.

🔗 CHECKPOINT QUESTION

A colleagues says: 'I added indexes on every column of the orders table — now all my SELECT queries run instantly but our nightly ETL that inserts 2 million rows takes 6 hours instead of 40 minutes.' Diagnose the problem and explain what you would do to fix it.

PRACTICE PROBLEMS — INDEX DESIGN AND AUDIT

ChatGPT • AI Assistant

A four-step process diagram shown as horizontal pipeline stages connected by arrows. Stage 1 (blue box, label 'IDENTIFY SLOW QUERIES'): shows EXPLAIN ANALYZE output with 'Seq Scan' highlighted in red and actual execution time of 45,823ms. Stage 2 (orange box, label 'ADD TARGETED INDEXES'): shows CREATE INDEX statement with annotation checklist — 'Leftmost = most selective column. INCLUDE = SELECT-only columns. Partial = WHERE subset only.' Stage 3 (green box, label 'VERIFY IMPROVEMENT'): shows EXPLAIN ANALYZE output after index with 'Index Only Scan' highlighted in green and actual execution time of 0.08ms. Stage 4 (purple box, label 'AUDIT AND DROP UNUSED'): shows pg_stat_user_indexes query result with columns indexname and idx_scan — three indexes with idx_scan=0 highlighted in red with label 'Zero scans since reset — candidates to drop.' A rule strip at the bottom reads: 'Never add an index without EXPLAIN before. Never keep an index with zero scans. Measure, don't guess.' Clean four-stage pipeline layout, consistent color coding, monospace SQL in each stage box.

Index Audit Workflow — Finding and Removing Harmful Indexes

sql – Practice Problems – Complete Index Design and Audit Solutions

```
-- =====
-- PROBLEM 1: Add correct indexes for three slow queries
-- =====

-- Query A: customer_id equality + order_date range + covering SELECT cols
CREATE INDEX idx_orders_cust_date_cover
  ON orders(customer_id, order_date DESC)
  INCLUDE (total_amount, status);
```

```
-- customer_id first: high cardinality equality → selective
-- order_date second: range within customer's rows
-- INCLUDE: covers SELECT columns → Index Only Scan

-- Query B: delivered orders in 2024 grouped by city
CREATE INDEX idx_orders_delivered_date_city
  ON orders(order_date, city)
  WHERE status = 'delivered';
-- PARTIAL: only delivered orders → smaller index
-- order_date first: range query entry point
-- city second: GROUP BY column follows

-- Query C: already covered by idx_orders_cust_date_cover
-- Verify:
EXPLAIN ANALYZE
SELECT customer_id, total_amount, status FROM orders WHERE customer_id
= 101;

-- =====
-- PROBLEM 2: Identify 3 indexes hurting write performance
-- =====

-- DROP #1: duplicate of PRIMARY KEY (redundant)
DROP INDEX idx_customers_pk;
-- customer_id PK already has an auto-created index

-- DROP #2: boolean column – low cardinality, optimizer ignores
DROP INDEX idx_customers_is_active;
-- Only 2 distinct values → ~50% selectivity → seq scan is faster

-- DROP #3: composite (name, email) – redundant with existing indexes
DROP INDEX idx_customers_name_email;
-- email queries → use idx_customers_email (more selective)
-- name queries → use idx_customers_name
-- name+email combo queries are rare and both single indexes cover them

-- BONUS DROP: idx_customers_city (redundant with
idx_customers_city_date)
DROP INDEX idx_customers_city;
-- idx_customers_city_date covers WHERE city = 'X' via leftmost prefix
rule

-- AUDIT QUERY: find unused indexes
SELECT indexname,
       idx_scan AS times_used,
       pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
```

```
FROM pg_stat_user_indexes
WHERE tablename = 'customers'
ORDER BY idx_scan ASC, pg_relation_size(indexrelid) DESC;

-- FINAL HEALTHY INDEX SET for customers:
-- idx_customers_email      (high cardinality, auth lookups)
-- idx_customers_name       (search feature)
-- idx_customers_city_date  (city + date range composite)
-- idx_customers_signup_date (date range queries)
```

Student

Can we go through the two practice problems — add the right index and audit a table with too many indexes?

ChatGPT • AI Assistant

CONCEPT CARD

These two practice problems represent the most common real-world indexing tasks: deciding what to add when a query is slow, and deciding what to drop when writes are suffering.

Practice Problem 1 — Given a slow query and schema, decide which index to add:

Table schema:

sql – Code Example

```
CREATE TABLE orders (
  order_id      SERIAL          PRIMARY KEY,
  customer_id   INT             NOT NULL REFERENCES
customers(customer_id),
  order_date    DATE            NOT NULL,
  total_amount  DECIMAL(12,2),
  status        VARCHAR(20)     NOT NULL DEFAULT 'pending',
  city          VARCHAR(50)
);
-- Current indexes: PRIMARY KEY on order_id only
-- Row count: 10,000,000
```

Slow queries to fix:

sql – Code Example

```
-- Query A: runs in 45 seconds
SELECT order_id, total_amount, status
FROM orders
WHERE customer_id = 101
    AND order_date > '2024-01-01'
ORDER BY order_date DESC
LIMIT 50;

-- Query B: runs in 38 seconds
SELECT city, COUNT(*) AS order_count, SUM(total_amount) AS revenue
FROM orders
WHERE status = 'delivered'
    AND order_date ≥ '2024-01-01'
    AND order_date < '2025-01-01'
GROUP BY city;

-- Query C: runs in 12 seconds on a hot path (called 10,000 times/sec)
SELECT customer_id, total_amount, status
FROM orders
WHERE customer_id = 101;
```

Solution — Index design decisions:

sql – Code Example

```
-- Query A fix:
-- customer_id is the selective filter (high cardinality) → leftmost
-- order_date is the range filter → second column
-- INCLUDE total_amount and status for covering (SELECT needs them)
CREATE INDEX idx_orders_cust_date_cover
    ON orders(customer_id, order_date DESC)
    INCLUDE (total_amount, status);

-- Why customer_id first?
-- customer_id = 101 is an equality filter → narrows to one customer's
rows
-- order_date is then a range within those rows → traversal is
efficient
-- If order_date were first, the index would cover ALL dates before
narrowing by customer

-- Why INCLUDE total_amount, status?
-- SELECT needs these columns. INCLUDE puts them in leaf nodes.
```

```

-- Query becomes an Index Only Scan – zero table fetches.

-- EXPLAIN ANALYZE after adding:
-- Index Only Scan using idx_orders_cust_date_cover
-- actual time: 0.08ms vs 45,000ms before

-- Query B fix:
-- status = 'delivered' is low cardinality BUT combined with date
range it helps
-- order_date is the range – put after the equality filter
-- Use a PARTIAL INDEX to index only delivered orders (smaller,
faster)
CREATE INDEX idx_orders_delivered_date_city
  ON orders(order_date, city)
  WHERE status = 'delivered';

-- Why partial index here?
-- We only ever query WHERE status = 'delivered' for this pattern
-- Partial index is smaller (only delivered rows) → fits in memory
better
-- city is in GROUP BY and SELECT – consider INCLUDE if index-only
scan needed

-- Query C fix:
-- Simple equality on customer_id, needs customer_id + total_amount +
status
-- Covered by idx_orders_cust_date_cover already created for Query A
-- No additional index needed – check EXPLAIN to confirm
EXPLAIN ANALYZE
SELECT customer_id, total_amount, status
FROM orders WHERE customer_id = 101;
-- Should show: Index Only Scan using idx_orders_cust_date_cover

```

Practice Problem 2 — Given a table with 8 indexes, identify the 3 that are hurting write performance and should be dropped:

Table: **customers** (500,000 rows)

Existing indexes:

sql – Code Example

```

idx_1: CREATE INDEX idx_customers_pk          ON
customers(customer_id);          -- PK duplicate
idx_2: CREATE INDEX idx_customers_email      ON customers(email);
-- high cardinality

```

```
idx_3: CREATE INDEX idx_customers_city          ON customers(city);
-- low cardinality (~50 cities)
idx_4: CREATE INDEX idx_customers_is_active      ON
customers(is_active);          -- boolean
idx_5: CREATE INDEX idx_customers_name          ON customers(name);
-- medium cardinality
idx_6: CREATE INDEX idx_customers_city_date     ON customers(city,
signup_date); -- composite
idx_7: CREATE INDEX idx_customers_signup_date  ON
customers(signup_date);        -- date range queries
idx_8: CREATE INDEX idx_customers_name_email   ON customers(name,
email);          -- composite
```

Solution — Identify the 3 worst indexes:

sql – Code Example

```
-- INDEX TO DROP #1: idx_customers_pk (duplicate of PRIMARY KEY)
-- The PRIMARY KEY on customer_id already creates an index
automatically.
-- idx_customers_pk is a redundant copy of the same index.
-- Write cost: doubles every PK update for zero benefit.
DROP INDEX idx_customers_pk;

-- INDEX TO DROP #2: idx_customers_is_active (boolean – low
cardinality)
-- is_active has only 2 distinct values (TRUE / FALSE).
-- WHERE is_active = TRUE returns ~50% of the table.
-- The optimizer ignores this index for most queries and uses a seq
scan.
-- Write cost: every INSERT and UPDATE to is_active maintains this
useless index.
DROP INDEX idx_customers_is_active;

-- INDEX TO DROP #3: idx_customers_name_email (redundant composite)
-- We already have idx_customers_email (single column) for email
lookups.
-- We already have idx_customers_name (single column) for name
lookups.
-- idx_customers_name_email adds write overhead with very limited
extra benefit:
--   WHERE name = X AND email = Y is a very rare query pattern
--   WHERE email = 'x@x.com' already uses idx_customers_email (more
selective)
-- It also overlaps partially with idx_customers_name for name-only
queries
```

```

-- but the optimizer will prefer the single-column name index for
those
DROP INDEX idx_customers_name_email;

-- INDEXES TO KEEP AND WHY:
-- idx_customers_email: high cardinality, used for login/auth lookups
-- critical
-- idx_customers_city: low cardinality BUT idx_customers_city_date is
better;
-- consider dropping idx_customers_city too (idx_6 covers it via
leftmost prefix)
-- idx_customers_name: medium cardinality, used for search features –
keep
-- idx_customers_city_date: composite for city + date range queries –
keep
-- idx_customers_signup_date: date range queries – keep

-- BONUS: idx_customers_city is also droppable (idx_6 covers city
queries)
DROP INDEX idx_customers_city;
-- With idx_customers_city_date in place:
-- WHERE city = 'Delhi' uses idx_customers_city_date (leftmost column)
-- Making idx_customers_city redundant

-- VERIFY the audit findings with usage stats:
SELECT indexname,
       idx_scan          AS times_used_since_reset,
       pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE tablename = 'customers'
ORDER BY idx_scan ASC; -- lowest usage first = candidates to drop

```

SUMMARY CARD

SQL 1.7 Key Takeaways:

- **Full sequential scan** = $O(N)$ — reads every row. Catastrophic on large tables.
- **B-tree index** = $O(\log N)$. Supports equality, range, prefix LIKE, ORDER BY. Default for all cases.
- **Hash index** = $O(1)$ equality only. Cannot support range, LIKE, ORDER BY. Use only for high-cardinality equality-only columns.
- **B-tree supports:** =, >, <, >=, <=, BETWEEN, LIKE 'abc%', ORDER BY.
- **B-tree does NOT support:** LIKE '%abc', functions on indexed columns, implicit type casts.

- **Composite index leftmost prefix rule:** INDEX(a, b) helps WHERE a or WHERE a AND b. Does NOT help WHERE b alone.
- **Covering index:** includes all SELECT columns in leaf nodes. Enables index-only scan — never touches base table.
- **Write cost:** every index adds one B-tree write per INSERT/UPDATE/DELETE. 10 indexes = 10x write overhead.
- **Do NOT index:** booleans, low-cardinality columns, tiny tables, SELECT-only columns.
- **Audit with:** EXPLAIN ANALYZE (before/after), *pgstatuser_indexes* (find unused indexes).
- **Always verify** index is actually used with EXPLAIN ANALYZE — optimizer may choose seq scan anyway.

🔮 AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT for deeper practice:

'Act as a senior database performance engineer. I just learned SQL indexes — B-tree, hash, composite, covering, and partial indexes. Give me 8 progressively harder indexing problems. Each problem should give me: a table schema, row counts, 2-3 slow queries, and existing indexes. Ask me to: diagnose why each query is slow using EXPLAIN output, decide which indexes to add, and identify any existing indexes to drop. After I answer: show the optimal solution, explain every indexing decision, show the expected EXPLAIN output change, and give one follow-up question about edge cases (null handling in indexes, partial indexes, index bloat). Do not continue until I submit my answer.'

🔗 CHECKPOINT QUESTION

Final review — you are handed this table and told 'everything is slow, please fix it':

sql – Code Example

```
CREATE TABLE events (  
    event_id    BIGINT PRIMARY KEY,  
    user_id     INT NOT NULL,  
    event_type  VARCHAR(30) NOT NULL, -- 8 distinct values  
    created_at  TIMESTAMPTZ NOT NULL,  
    session_id  UUID NOT NULL,  
    payload     JSONB  
);  
-- Row count: 500,000,000 (500 million rows)  
-- Existing indexes: PRIMARY KEY on event_id only
```

```
-- Slow queries:
-- Q1: WHERE user_id = 12345 ORDER BY created_at DESC LIMIT 20
-- Q2: WHERE session_id = 'abc-def-...' (exact UUID lookup)
-- Q3: WHERE event_type = 'page_view' AND created_at > NOW() -
INTERVAL '7 days'
-- Q4: SELECT user_id, COUNT(*) FROM events
--       WHERE created_at > NOW() - INTERVAL '30 days'
--       GROUP BY user_id HAVING COUNT(*) > 100
```

Design the minimum set of indexes to fix all four queries. Write each CREATE INDEX statement, name it, and justify every decision including the index type (B-tree vs hash), column order in composites, INCLUDE columns for covering, and whether a partial index applies.

✨ CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including sql 1.7 — indexes: b-tree and hash, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Full Sequential Scan vs Index Lookup — The Core Trade-Off:** A two-panel comparison diagram. Left panel (red background, label 'WITHOUT INDEX...')